Empirical Studies in Software Engineering Involving Large Language Models

Davide Falessi, PhD

Università di Roma Tor Vergata

Corso di Ingegneria del Software 2

Agenda

- Contesto
- Ruolo degli LLM in SE
- Linee guida per progettare e riportare studi SE basati su LLM
- Sfide

Introduzione

- Gli LLM sono ormai centrali nella ricerca e nella pratica del software engineering.
- Tuttavia, il loro uso introduce nuovi **problemi metodologici**.
- Questa presentazione propone una tassonomia degli studi e un insieme di linee guida per progettare e presentare studi empirici in modo più rigoroso.

Problema

- Si osserva come molti studi empirici con LLM presentino **bassa riproducibilità**.

- Le cause principali sono:
  - **non-determinismo,**
  - **rapida evoluzione dei modelli commerciali,**
  - **scarsa trasparenza su training data e dettagli di fine-tuning.**

Soluzione

- In uno studio empirico con LLM bisogna documentare:
  - **quale modello,**
  - **quale versione,**
  - **quale configurazione,**
  - **quali prompt,**
  - **quale architettura di sistema,**
  - **quali metriche,**
  - **quali limiti**.
- L'obiettivo delle linee guida non è eliminare il non-determinismo, ma rendere le conclusioni più verificabili, auditabili e confrontabili.

Come progettare e descrivere uno studio basato su LLM

- La presentazione si rivolge:
  - A ricercatori che progettano studi empirici con LLM.
  - A ricercatori che devono scrivere il paper finale.
  - A revisori che devono valutare la qualità metodologica dello studio.
  - Questo contenuto non sostituisce le linee guida generali per la ricerca empirica, ma le integra per i problemi specifici degli LLM.

Metodologia

- Il lavoro nasce da una collaborazione tra 22 ricercatori.
- È stato sviluppato attraverso incontri periodici, raffinamento iterativo e revisione tra pari.
- Gli autori hanno anche raccolto esempi dalla letteratura per illustrare i diversi tipi di studio e le linee guida.
- Quindi non è una semplice posizione individuale, ma un tentativo di standardizzazione comunitaria.

Tipologie di studi (ruolo del LLM)

- S1: LLMs as Annotators
- S2: LLMs as Judges
- S3: LLMs for Synthesis
- S4: LLMs as Subjects
- S5: Studying LLM Usage in Software Engineering
- S6: LLMs for New Software Engineering Tools
- S7: Benchmarking LLMs for Software Engineering Tasks

Domande di ricerca

- S1: possiamo fidarci degli LLM come annotatori?
- S2: possiamo usare gli LLM come giudici?
- S3: gli LLM possono supportare la sintesi qualitativa senza perdere sfumature?
- S4: gli LLM possono simulare sviluppatori o utenti?
- S5: come cambiano i comportamenti dei programmatori quando usano LLM?
- S6: come progettare tool LLM-based realmente utili?
- S7: i benchmark attuali misurano davvero la capacità di software engineering?

S1: LLMs as Annotators

- Qui l'LLM aiuta a fare coding o labeling di dati qualitativi:
  - requisiti,
  - transcript di interviste,
  - survey aperte,
  - issue report.
- Il vantaggio potenziale è ridurre il lavoro manuale.
- Il problema è che l'annotazione non è soltanto una procedura tecnica, ma può anche essere un'interpretazione.
- E.g.: dato un ticket, deciderne la tipologia (bug o feature). Data una classe, decidere se è buggy o meno.

S2: LLMs as Judges

- In questo caso l'LLM non etichetta il testo libero, ma valuta la qualità o le proprietà di un artefatto. Esempi:
  - qualità di requisiti,
  - leggibilità del codice,
  - qualità di commenti,
  - aderenza a standard.

S3: LLMs for Synthesis

- Qui l'LLM serve a sintetizzare, non solo a classificare.

- Significa integrare più fonti per:
  - individuare pattern,
  - costruire temi,
  - produrre insight più astratti,
  - supportare analisi qualitative.

- Esempio:
  - Systematic literature review o systematic mapping study

S4: LLMs as Subjects

- L'LLM viene usato come "partecipante virtuale".

- Può simulare:
  - sviluppatori junior vs senior,
  - utenti di un sistema,
  - candidati in un processo di recruitment,
  - partecipanti ad interviste o focus group.

- Esempio: LLM viene usato come sviluppatore per confrontare due stili architetturali (ad es. EBC vs. incidental).

S5: Studying LLM Usage in Software Engineering

- Qui non si studia il modello in sé, bensì come gli sviluppatori usano gli LLM.

- Metodi tipici:
  - survey,
  - interviste,
  - osservazione sul campo,
  - analisi dei log,
  - mining di conversazioni.

- Esempio: osservare come gli LLM vengono usati da un gruppo di sviluppatori e misurare il tempo impiegato, la tipologia delle query, il numero di cambiamenti, ecc.

S6: LLMs for New Software Engineering Tools

- In questo tipo di studio si costruisce un nuovo strumento che integra LLM.

- Esempi:
  - code explanation,
  - test generation,
  - bug fixing assistant,
  - agenti di sviluppo.

S7: Benchmarking LLMs for Software Engineering Tasks

- Qui si confrontano LLM su task standardizzati, metriche standardizzate e dataset standardizzati.
- Task tipici:
  - code generation,
  - summarization,
  - completion,
  - code repair.

- Esempio: confrontare le versioni gratuite di modelli proprietari (e.g., Gemini, Copilot, Claude etc) con modelli Open (e.g., Llama) per eseguire il task X (e.g., refactoring).
- Metriche: pass rate sui test, tempo medio, stabilità su ripetizioni multiple.

Linee guida

- G1: dichiarare uso e ruolo dell'LLM
- G2: riportare versione, configurazione e customizzazioni
- G3: riportare l'architettura del tool oltre il modello
- G4: riportare prompt, sviluppo dei prompt e interaction logs
- G5: usare validazione umana
- G6: usare un open LLM come baseline
- G7: usare baseline, benchmark e metriche adeguate
- G8: riportare limitazioni e mitigazioni

Linee guida

Table 3 Applicability of guidelines to study types. ● = the guideline's core recommendations MUST be followed for this study type, ◌ = SHOULD be followed, – = are not directly applicable. Study type IDs: S1 = *LLMs as Annotators*, S2 = *LLMs as Judges*, S3 = *LLMs for Synthesis*, S4 = *LLMs as Subjects*, S5 = *Studying LLM Usage*, S6 = *LLMs for Tools*, S7 = *Benchmarking LLMs*. Each guideline's study-type-specific guidance is detailed in the corresponding subsection.

	S1	S2	S3	S4	S5	S6	S7
G1: Declare Usage	●	●	●	●	●	●	●
G2: Model Version	●	●	●	●	●	●	●
G3: Architecture	◌	◌	◌	◌	◌	●	◌
G4: Prompts	●	●	●	●	●	●	●
G5: Human Valid.	◌	◌	◌	◌	◌	◌	◌
G6: Open LLM	◌	◌	◌	–	–	◌	●
G7: Benchmarks	●	◌	◌	◌	◌	◌	●
G8: Limitations	●	●	●	●	●	●	●

G1: DICHIARARE USO E RUOLO DELL'LLM

- Se usi un LLM in uno studio empirico, devi dirlo chiaramente. E' necessario specificare:
  - quale LLM,
  - come è stato usato,
  - in quale fase del processo,
  - per quale scopo,
  - con quali benefici attesi.
  - Se l'LLM ha un ruolo centrale, la disclosure deve stare in modo prominente nella metodologia.
- Esempio
  - Cattivo reporting: "Abbiamo usato ChatGPT per aiutarci nell'analisi."
  - Buon reporting: "Abbiamo usato GPT-4o come annotatore preliminare dei ticket bug/non-bug nella fase di preprocessing, con l'obiettivo di ridurre il carico di annotazione umana."

G2: RIPORTARE VERSIONE, CONFIGURAZIONE E CUSTOMIZZAZIONI

- Informazioni da riportare:
  - nome del modello,
  - versione,
  - data dell'esperimento,
  - parametri di generazione,
  - eventuale seed,
  - contesto/finestra di token,
  - quantizzazione, se presente,
  - dettagli di fine-tuning, se applicabili.

- Un esempio raccomandato include: modello (ad es. GPT-4 su Azure), temperatura, top_p, max token, seed, data e fingerprint del sistema.

G3: DESCRIVERE L'ARCHITETTURA DEL TOOL OLTRE IL MODELLO

- Un tool LLM-based non coincide con il modello.
- Informazioni da riportare:
  - preprocessing,
  - retrieval,
  - prompt construction,
  - parsing dell'output,
  - retry logic,
  - interazione con database e API,
  - hosting,
  - hardware,
  - latenza.

G3 NEI SISTEMI **AGENTICI**

- Informazioni da riportare:
  - **Ruoli degli agenti**,
  - **Architettura:** E.g, planner/executor/coordinator,
  - **Reasoning framework**: E.g.,:
    - **ReAct (Reason + Act):** L'agente esplicita un pensiero ("Devo cercare X"), compie un'azione (chiama un motore di ricerca), osserva il risultato e poi ripete il ciclo finché non ha la risposta.
    - **Chain of Thought (CoT):** L'agente è costretto a spiegare tutti i passaggi logici prima di fornire la risposta finale.
    - **Reflection / Self-Correction**: L'agente analizza il proprio output precedente per individuare eventuali errori e correggerli in autonomia.
  - **Interazione con strumenti esterni**: elenco esatto delle API, dei database o dei software a cui l'agente ha accesso.
  - **Tracciamento completo delle azioni**. E.g., azioni di un agente di bug fixing: e.g., legge l'issue, cerca file rilevanti, modifica il codice, lancia i test, propone la patch.

G4: RIPORTARE PROMPT, LORO SVILUPPO E INTERACTION LOGS

- I prompt sono parte del metodo sperimentale.
- Informazioni da riportare:
  - prompt completi o template,
  - componenti statiche e dinamiche,
  - formattazione,
  - esempi few-shot,
  - strategia di prompting,
  - evoluzione dei prompt,
  - interaction logs, quando possibile.
- Un prompt non è un dettaglio secondario: può cambiare in modo sostanziale la qualità del risultato.

G5: USARE VALIDAZIONE UMANA

- Quando il costrutto da valutare è soggettivo o complesso, le metriche automatiche non bastano.
- Serve confronto con giudici umani.
- Un alto accordo tra LLM non implica validità.
  - Anche un alto accordo umano non implica automaticamente la validità, ma senza validazione umana per certi task non sappiamo davvero cosa stiamo misurando.
- Bisogna:
  - definire il costrutto,
  - descrivere lo strumento di misura,
  - riportare agreement,
  - spiegare perché il giudizio umano è plausibilmente valido.

- Esempio: qualità dei requisiti prodotti.

G6: USARE UN OPEN LLM COME BASELINE

- Quando si usano modelli commerciali in modo centrale, è raccomandabile **includere anche un modello open**.
  - Questo migliora la replicabilità, almeno parzialmente, dello studio.

- Serve anche un replication package con istruzioni chiare.

- Esempio**:** Se confronto solo GPT-4.1 e Claude, sto conducendo uno studio interessante ma difficile da replicare. Se aggiungo Llama o un altro modello open, una parte del lavoro diventa molto più verificabile.

G7: USARE BASELINE, BENCHMARK E METRICHE ADEGUATE

- Le metriche non vanno scelte “perché le usano tutti”.
- Bisogna spiegare:
  - cosa si vuole misurare,
  - perché quella metrica è appropriata,
  - quali limiti ha,
  - quali benchmark sono adatti al contesto.

- La diffusione di una metrica non implica la validità del costrutto.
- Esempi di metriche:
  - Code generation: BLEU, CodeBLEU, pass@k, Exact Match.
  - Recommendation: Mean Reciprocal Rank.
  - Classificazione: …

G7: RIPETIZIONI E STATISTICA

- Poiché gli LLM sono non-deterministici, gli esperimenti dovrebbero essere ripetuti.

- Vanno riportate statistiche descrittive:
  - media,
  - mediana,
  - deviazione standard,
  - intervalli di confidenza.

- Se si confrontano modelli o tool, servono anche statistiche inferenziali appropriate.

G8: RIPORTARE LIMITAZIONI E MITIGAZIONI

- Ogni studio deve discutere apertamente:
  - limiti,
  - minacce alla validità,
  - mitigazioni.
- Non basta una sezione finale standard e generica. La sezione limitazioni deve essere specifica per il tipo di studio e le scelte fatte.

G8: ESEMPI DI LIMITAZIONI DA DISCUTERE

- Validità interna: leakage, contaminazione, pipeline non documentata.
- Validità di costrutto: metrica che non misura davvero il concetto di interesse.
- Validità esterna: risultati che non generalizzano ad altri modelli, tempi, contesti o popolazioni.
- Riproducibilità: dipendenza da infrastruttura, modelli SaaS, costi elevati.
- Etica: dati sensibili, bias, privacy.
- Sostenibilità: ripetere molte run migliora l'affidabilità, ma aumenta consumo energetico.

Errori comuni

- “Abbiamo usato ChatGPT” senza specificare versione e parametri.
- Prompt non riportati.
- Nessuna indicazione su data dell’esperimento.
- Metriche scelte solo per consuetudine.
- Nessuna ripetizione sperimentale.
- Assenza di validazione umana in task soggettivi.
- Nessuna discussione su leakage o bias.
- Architettura del tool ridotta a una riga.

Sfida 1: riproducibilità, non-determinismo e instabilità nel tempo

- La prima grande sfida è il **non-determinismo**: prompt identici possono produrre output diversi tra run diverse.
- Piccole differenze nei prompt, nell'ordine delle opzioni o nei parametri possono cambiare in modo significativo i risultati.
- Anche i modelli commerciali possono cambiare nel tempo, oltre alla semplice etichetta di versione.
- Quindi replicare uno studio a distanza di settimane o mesi può diventare difficile anche usando "lo stesso" sistema.
- Conclusione: Il problema non è solo che il modello "sbaglia"; è che il sistema sperimentale può essere **mobile**, cioè cambiare mentre lo stiamo studiando.

RD: rendere uno studio replicabile. E.g., possiamo fare una snapshot (riusabile) dell'LLM? (e.g., sandbox)

Sfide 2: affidabilità, validità e bias

- Un modello LLM può concordare con sé stesso o con altri modelli e, tuttavia, risultare sistematicamente distorto.
- Esistono rischi di bias.
- **Affidabilità e validità sono due concetti diversi**.

RD: identificazione di bias e loro mitigazione nell'ambito del SE. Cambiano tra usi diversi (defect fix vs. feature implementation)?

Sfida 3: benchmark, metriche e costrutti

- Una metrica diffusa non è automaticamente valida.
- Esiste il rischio di metric–construct mismatch: la metrica misura una proprietà facile da contare, ma non il costrutto davvero importante.
- Esiste anche il rischio di una limitazione dello scope del benchmark: copre solo una porzione ristretta del lavoro effettivo di SE.

RD: creazione di nuove metriche per task eseguiti da LLM

Sfida 4: generalizzabilità, privacy, risorse e sostenibilità

- I risultati possono non generalizzare:
  - da un modello ad altri modelli,
  - da una configurazione ad altre,
  - dal laboratorio alla pratica,
  - da un periodo temporale a un altro,
  - da un campione di utenti a una popolazione più ampia.

RD: come posso progettare un esperimento in modo che il risultato sia generalizzabile? (e.g., usare uno specifico insieme di LLM)

Sfide 5: generalizzabilità, privacy, risorse e sostenibilità

- Studi con LLM possono essere limitati da:
  - dati sensibili o proprietari,
  - vincoli normativi,
  - dipendenza da infrastruttura,
  - disuguaglianza di risorse tra gruppi di ricerca,
  - costi energetici elevati.

RD: rendere gli studi più fattibili.

Gaps

- **Gap 1 — Valutazione empirica ancora fragile.** Metriche comuni non catturano sempre qualità SE rilevanti: correttezza, manutenibilità, sicurezza, leggibilità, robustezza, utilità. **Possibile ricerca:** confrontare metriche automatiche, test suite e giudizio umano.

- **Gap 2 — Benchmark poco realistici rispetto alla pratica SE.** Molti benchmark coprono task ristretti e poco rappresentativi del lavoro reale. **Possibile ricerca:** valutare o progettare benchmark multi-file, realistici e orientati alla qualità software.

- **Gap 3 — Riproducibilità e stabilità dei risultati.** Risultati sensibili a modello, versione, prompt, configurazione, infrastruttura e tempo. **Possibile ricerca:** replication study di studi LLM4SE con modelli, prompt o date diverse.

Gaps

- **Gap 4 — Pochi dati industriali e limitata generalizzabilità.** Molti studi usano dataset accademici; Hou et al. segnalano scarsità di dati industriali. **Possibile ricerca:** case study, esperimenti o osservazioni con sviluppatori reali.

- **Gap 5 — Ruolo umano ancora poco compreso**
  Non è chiaro quando l'LLM debba sostituire, assistere o essere controllato dall'umano. **Possibile ricerca:** studi human-in-the-loop su annotazione, giudizio, code review o agenti.

- **Gap 6 – Sistemi agentici**. Quasi tutto; e.g., adeguare processi e architetture software applicati a non umani. (e.g., Individuals and interactions over process and tools)

Referenze

- Anita D. Carleton, Davide Falessi, Hongyu Zhang, Xin Xia: ***Generative AI: Redefining the Future of Software Engineering***. IEEE Softw. 41(6): 34-37 (2024)

- Stefan Wagner, Marvin Muñoz Barón, Davide Falessi, Sebastian Baltes: ***Towards Evaluation Guidelines for Empirical Studies Involving LLMs***. WSESE@ICSE 2025: 24-27

- Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil A. Ernst, Davide Falessi, Brian Fitzgerald, Davide Fucci, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Lutz Prechelt, Paul Ralph, Christoph Treude, Stefan Wagner: ***Guidelines for Empirical Studies in Software Engineering involving Large Language Models***. CoRR abs/2508.15503 (2025)